

v0.4 코드 복습 노트

2026-10-10 야간 · 약 2시간 · 인터넷 없이 · 내 앱(I will become monster)의 진짜 코드로

0. 오늘 복습 쓰는 법

5분

🕒 2시간 뒤에 "버튼 하나를 누르면 무슨 일이 일어나는지" 처음부터 끝까지 내 말로 설명할 수 있기

- 순서대로 가되, 막히면 ☆ 표시하고 넘어가. 다 끝나고 ☆만 다시 보면 돼.
- 문제는 먼저 **예상**을 적고(종이나 메모) → 그다음 정답(맨 아래 정답)을 봐. 예상 없이 정답부터 보면 남는 게 거의 없어.
- **실행 셀**: PC에서 이 파일을 열었으면 ▶ **실행**을 눌러 바로 결과를 볼 수 있어 (인터넷 필요 없음). 크롬 F12 → Console에 붙여넣어도 돼. 폰 PDF에서는 실행 대신 예상만 하고 정답과 비교해.
- 코드는 전부 **지금 네 앱의 진짜 코드**에서 가져왔어. 길면 줄였고, 줄인 곳은 ...로 표시했어.

파트	시간	핵심
☐ 1. 앱 지도	15분	파일 9개 · data와 ui · 큰 흐름 한 장
☐ 2. JS 문법 다시 잡기	25분	Python 짝 맞추기 · map/filter/reduce · ?? · 참/거짓
☐ 3. HTML은 어떻게 그려지나	20분	태그 · render() · 템플릿 글자 · join · escapeHtml · CSS 토큰
☐ 4. 버튼이 함수에 닿기까지	15분	data-action · closest · dataset · 클릭 따라가기
☐ 5. 저장과 동기화	20분	save/load · JSON · async/await · 올리기 → 받기 · 서비스 워커
☐ 6. 핵심 규칙 3개 손으로	15분	다음 무게 · 유지 칼로리 · 그래프 점
☐ 7. 고칠 곳 찾기	15분	"이걸 바꾸려면 어느 파일의 무엇?" 12문제
☐ 8. 조용한 버그 도감 + 말로 설명하기	10분	에러 없이 틀리는 것들 총정리

1. 앱 지도 — 파일 9개와 큰 흐름

15분

🕒 어떤 걸 고치고 싶을 때 "어느 파일을 열지" 바로 떠올리기

1-1. 파일마다 하는 일

파일	하는 일	대표 이름
index.html	빠대. 빈 상자 하나(#app) + 스크립트를 위에서부터 순서대로 불러옴	<div id="app">
style.css	모양. 색은 토큰 (--accent 등)으로 한 곳에서 관리, 다크 모드도 여기	--accent
exercises.js	기본 종목 설정표 (노트북 ②와 같음)	EXERCISES
app.js	저장(data · save · load) + 운동 기록 화면 + render() + 버튼 연결의 중심	data, ui, render, nextWeight
nutrition.js	체중 · 식단: 계산(노트북 v0.3) + 화면 + 동기화	maintenanceInfo, dayTotals
progress.js	운동 그래프 + 숨기기	progressSeries, elrm
import.js	메모 가져오기 (노트북 ③ parse_memo)	parseMemo
sync.js	로그인 + 올리기 · 받기 (Supabase)	syncNow, uploadWorkout
sw.js	서비스 워커: 앱 파일을 폰에 보관 → 인터넷 없이도 열림	CACHE

+ 저장소의 supabase/migrations/*.sql = 인터넷 표 설계도 (표 · 칸 · RLS).

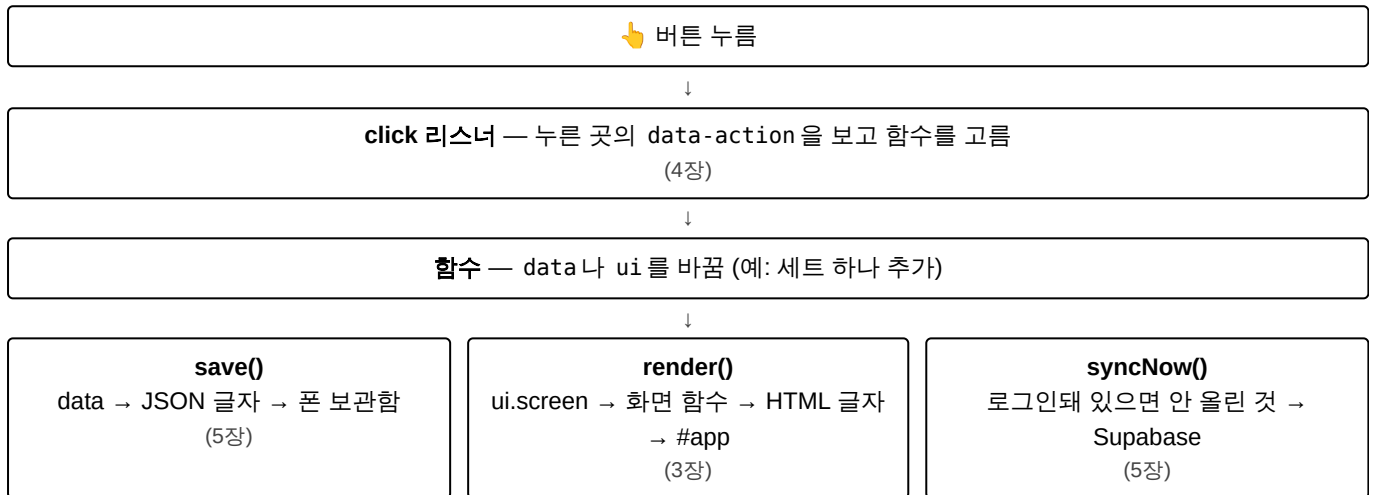
1-2. 상자 두 개: data와 ui

app.js (줄임)

```
let data = load();           // 기록 (운동 · 체중 · 식사 · 설정) → 저장함. 껴다 켜도 남음
const ui = {                 // 지금 화면 상태 → 저장 안 함. 껴다 켜면 처음으로
  screen: "home",           // 어느 화면? home / workouts / workout / nutrition ...
  workoutTab: "log",        // Workout 화면의 탭
  active: null,             // 펼쳐진 종목 번호
  ...
};
// nutrition.js · progress.js는 자기 칸을 덧붙임: Object.assign(ui, { day: null, mealForm: null, ... })
```

Python으로 치면 data는 파일에 저장하는 dict, ui는 프로그램이 켜져 있는 동안만 쓰는 dict야.

1-3. 앱 전체가 도는 고리 (이 그림 하나가 핵심)



앱의 거의 모든 버튼이 이 고리를 돌아. "바꾸고 → 저장하고 → 다시 그리고 → (올리고)".

문제 — 먼저 종이에 예상하고, 정답은 맨 아래

Q1-1 아래 4개는 data와 ui 중 어디에 있을까? (가) 오늘 체중 83.6 (나) 지금 보고 있는 탭(식단/체중) (다) 펼쳐둔 그래프 (라) 숨긴 종목 목록

Q1-2 앱을 완전히 껴다 켜면 Q1-1의 (가)~(라) 중 무엇이 처음 상태로 돌아갈까?

Q1-3 다음을 고치려면 어느 파일을 열까? (가) 운동 그래프 점 계산 (나) 로그인 에러 문구 (다) Bench press 반복 범위 (라) 버튼 파란색

2. JS 문법 다시 잡기 — Python 짝 맞추기

25분

🔗 앱 코드에 자주 나오는 문법 20개를 Python으로 바꿔 읽을 수 있기

2-1. 짝 맞추기 표 (이것만 알아도 앱 코드의 80%가 읽혀)

JS	메모	Python
const x = 1	let x = 1	x = 1 — const는 다시 못 넣음, let은 다시 넣을 수 있음

JS	메모	Python
function f(a) { return a * 2; }		def f(a): return a * 2
a => a * 2	화살표 함수	lambda a: a * 2
`\${w}kg × \${r}회`	백틱 + \${}	f"\${w}kg × {r}회"
=== / !==		== / !=
&& / / !		and / or / not
a ?? b	a가 null/undefined일 때만 b	b if a is None else a
조건 ? a : b		a if 조건 else b
for (const s of sets)		for s in sets:
list.map(s => s.r)		[s['r'] for s in list]
list.filter(s => s.w === 85)		[s for s in list if s['w'] == 85]
list.find(s => s.r >= 10)	없으면 undefined	next((s for s in list if ...), None)
list.some(f) / list.every(f)		any(...) / all(...)
list.reduce((sum, x) => sum + x, 0)	0 = 시작값	sum(list)
Object.keys(d) / Object.entries(d)		list(d.keys()) / list(d.items())
[...a] / { ...d, k: 1 }	펼치기	a.copy() / {**d, 'k': 1}
const [c, p, f] = [50, 30, 20]	꺼내기	c, p, f = [50, 30, 20]
new Set(list)		set(list)
Math.max(...list)	... 필수!	max(list)
list.length		len(list)
null / undefined	없음 / 그런 칸 없음	None

2-2. 목록 다루기 — 세트 목록으로 연습

앱에서 세트는 { w: 무게, r: 횟수 } 모양이야. 아래 ①~⑤의 출력을 먼저 예상해 봐.

실행 셀 2-1

```
const sets = [{ w: 85, r: 8 }, { w: 85, r: 7 }, { w: 80, r: 10 }];

console.log(sets.map(s => s.r));           // ① 횟수만 뽑기
console.log(sets.filter(s => s.w === 85).length); // ② 85kg 세트 개수
console.log(sets.find(s => s.r >= 10));      // ③ 처음으로 10회 이상인 세트
console.log(sets.some(s => s.r < 8), sets.every(s => s.r >= 8)); // ④
console.log(Math.max(...sets.map(s => s.w))); // ⑤ 가장 무거운 무게
```

2-3. 사전 다루기 · reduce의 시작값

실행 셀 2-2

```
const kcal = { "10-08": 2200, "10-09": 1130 };

console.log(Object.keys(kcal)); // ①
console.log(Object.entries(kcal).map(([day, k]) => day + "=" + k).join(", ")); // ②
console.log(Object.keys(kcal).reduce((sum, d) => sum + kcal[d], 0)); // ③
console.log(Object.keys(kcal).reduce((sum, d) => sum + kcal[d])); // ④, 0 없음
console.log([...new Set(["Dips", "Pull up", "Dips"])]); // ⑤
```

2-4. 펼치기(...) · 꺼내기 · ?? · &&

...은 "안에 있는 걸 펼쳐 놓기"야. 같은 이름이 또 나오면 뒤에 온 게 이겨(덮어씀). 앱의 load()가 이걸로 예전 저장본에 없는 칸을 채워 (5장).

실행 셀 2-3

```
const saved = { workouts: [1, 2], custom: {} };
const d = { deleted: [], body: {}, ...saved, custom: { A: 1 } };
console.log(Object.keys(d), d.custom); // ① 펼치기(...) 순서와 덮어쓰기

const [c, p, f] = [50, 30, 20];
console.log(p); // ② 꺼내기

let draft = null;
console.log(draft ?? 70, 0 ?? 70, 0 || 70); // ③ ?? 와 || 의 차이

const rec = null;
console.log(rec && rec.w); // ④
```

2-5. 참/거짓 — Python과 다른 곳 ⚠

Python에서는 not [] 이 True(빈 목록 = 거짓)였지? JS는 다르니까 조심.

실행 셀 2-4

```
function check(x) {
  if (x) return "참";
  return "거짓";
}
console.log(check(0), check(""), check(null)); // ①
console.log(check([], check({}), check("0"))); // ② 파이썬과 다른 곳!
console.log(check([].length)); // ③
```

그래서 앱에서는 빈 목록인지 볼 때 꼭 .length를 써: if (!items.length), items.length ? ... : "아직 기록이 없어요".

문제 — 먼저 종이에 예상하고, 정답은 맨 아래

Q2-1 실행 셀 2-1의 ①~⑤ 출력을 예상해 봐.

Q2-2 실행 셀 2-2의 ①~⑤ 출력을 예상해 봐. 특히 ④는 왜 그렇게 나올까?

Q2-3 실행 셀 2-3의 ①~④ 출력을 예상해 봐. ③에서 0 ?? 70과 0 || 70은 왜 다를까?

Q2-4 실행 셀 2-4의 ①~③ 출력을 예상해 봐. 앱에서 if (!items.length) 라고 쓰는 이유는?

Q2-5 Python으로 바꿔 봐: `const top = Math.max(...sets.filter(s => s.r >= 8).map(s => s.w));`

3. HTML은 어떻게 그려지나 — render()와 템플릿 글자

20분

🔗 "화면 = JS가 만든 HTML 글자"라는 걸 코드로 따라가기

3-1. 태그 하나 해부하기

workoutsView가 만드는 '지난 운동' 버튼 하나

```
<button class="history" data-action="open-summary" data-id="a">10/05 등</button>
```

└ 태그 이름 └ 모양(CSS) └ 무슨 버튼인지(JS) └ 어느 운동 └ 내용 = 보이는 글자 └ 닫는 태그

- **class** → style.css의 규칙과 연결 (모양)
- **data-...** → JS가 읽는 메모 (4장). 화면에는 안 보임

3-2. index.html은 뼈대뿐 — 그리고 스크립트 순서

index.html (줄임)

```
<body>
  <div id="app"></div>
  <script src="exercises.js"></script>
  <script src="app.js"></script>
  <script src="nutrition.js"></script>
  <script src="progress.js"></script>
  <script src="vendor/supabase-2.117.2.js"></script>
  <script src="sync.js"></script>
  <script src="import.js"></script>
</body>
```

← 화면이 들어갈 빈 상자 하나
← 위에서부터 차례로 실행
← sync.js보다 먼저!
← 여기서 앱을 열자마자 동기화 시작

실제로 있었던 버그: 처음엔 nutrition.js가 sync.js 아래에 있었어. 앱을 열자마자 하는 동기화가 체중 · 식사 코드를 불러오기 전에 돌아서, 체중 · 식사가 안 올라갔지. 그래서 순서를 바꿨어. 또 다른 파일의 함수를 부를 때 if (typeof syncNutrition === "function")처럼 "있으면 부르기"로 써 둔 것도 같은 이유야.

3-3. render() — 화면 함수 고르기

app.js (줄임)

```
function render() {
  const views = { home: homeView, workouts: workoutsView, workout: workoutView, ...,
                  nutrition: () => nutritionView() }; // 이름 → 화면 함수
  document.getElementById("app").innerHTML = views[ui.screen](); // 고른 함수를 불러서 나온 HTML 글자를
  상자에 넣기
}
```

Python으로: `views = {"home": home_view, ...} → html = views[ui["screen"]]()`. 사전에 함수를 넣어 두고, 꺼내서 () 로 부르는 거야.

3-4. 화면 함수 = HTML 글자 만들기 · map + join

실행 셀 3-1

```
const past = [{ id: "a", split: "등", sets: 26 }, { id: "b", split: "가슴", sets: 44 }];

const list = past.map(w => `<button data-action="open-summary" data-id="${w.id}">${w.split} ${w.sets}세트</button>`);
console.log(list.length); // ① map의 결과는?
console.log(list.join("")); // ② 글자 하나로 붙이기
console.log(`<main>${list}</main>`); // ③ join 없이 바로 넣으면?
```

진짜 앱의 workoutsView() 도 똑같이 `past.map(w => `<button ...>`).join("")` 으로 목록을 만들어.

3-5. escapeHtml — 사용자가 적은 글자는 꼭 바뀌서 넣기

실행 셀 3-2

```
function escapeHtml(text) {
  return String(text).replace(/([<>"']/g, c => ({ "&": "&amp;", "<": "&lt;", ">": "&gt;", "'":
"&quot;", '"': "&#39;" }[c]));
}
const food = "<i>닭</i> 가슴살";
console.log(`<b>${food}</b>`); // ① 그대로 넣으면 (화면에서는 '닭'이 기울어짐)
console.log(`<b>${escapeHtml(food)}</b>`); // ② 바뀌서 넣으면 (화면에 글자 그대로 보임)
```

음식 이름, 종목 이름처럼 **사용자가 적은 글자**는 화면에 넣기 전에 `escapeHtml()` 을 거쳐. 안 그러면 글자가 HTML로 해석돼서 모양이 깨지거나, 나쁜 코드가 들어갈 수도 있어.

3-6. CSS — class와 색 토큰

style.css (줄임)

```
:root { --accent: #2a78d6; ... } /* 라이트 모드 색 */
@media (prefers-color-scheme: dark) {
  :root:not([data-theme="light"]) { --accent: #3987e5; ... } /* 폰이 다크 모드면 */
}
:root[data-theme="dark"] { --accent: #3987e5; ... } /* 다크로 고정했을 때 */

button.primary { background: var(--accent); color: var(--accent-text); } /* 토큰을 꺼내 쓰기 */
```

문제 — 먼저 종이에 예상하고, 정답은 맨 아래

Q3-1 실행 셀 3-1의 ①~③ 출력을 예상해 봐. ③처럼 `join("")` 없이 넣으면 화면에는 무엇이 더 보일까?

Q3-2 실행 셀 3-2의 ①, ②를 예상해 봐. 화면에서 ①과 ②는 각각 어떻게 보일까?

Q3-3 `render()` 에서 `ui.screen` 이 "nutrition" 이면 어떤 함수가 불릴까? 그 함수는 어느 파일에 있을까?

Q3-4 `nutrition.js` 줄을 다시 `sync.js` 아래로 옮기면, 앱을 열었을 때 무슨 일이 생길까? (에러가 날까?)

Q3-5 앱의 파란색을 초록색으로 바꾸려면 `style.css`에서 `--accent` 를 몇 군데 고쳐야 할까? 왜?

4. 버튼이 함수에 닿기까지 — data-action

15분

🕒 버튼 하나를 눌렀을 때 어떤 함수가 어떤 값으로 불리는지 따라가기

4-1. 리스너 하나가 모든 클릭을 받음

app.js (줄임)

```
document.addEventListener("click", event => { // 화면 어디를 눌러도 여기로 옴
  const el = event.target.closest("[data-action]"); // 누른 곳에서 위로 올라가며 data-action 달린 태그
  찾기
  if (!el) return; // 없으면 (빈 곳을 누름) 아무것도 안 함
  const i = Number(el.dataset.i), j = Number(el.dataset.j);
  switch (el.dataset.action) { // 이름 보고 나누기 (Python의 if/elif)
    case "start": return startWorkout(el.dataset.split);
    case "complete": return completeSet(i);
    case "home": ui.screen = "home"; return render();
    ""
  }
});
```

HTML에 쓴 것	JS에서 읽는 법	값
data-action="start"	el.dataset.action	"start"
data-split="가슴"	el.dataset.split	"가슴"
data-i="2"	el.dataset.i	"2" ← 글자! 그래서 Number()

closest가 필요한 이유: 버튼 안의 글자를 눌러도 event.target은 야. 거기서 위로 올라가며 data-action이 있는 버튼을 찾아 주는 게 closest야.

파일마다 리스너가 따로 있어: app.js(운동), nutrition.js(n-...), progress.js(graph-...), sync.js(로그인), import.js(가져오기). 모두 클릭을 듣지만, 각자 자기 이름만 처리하고 모르는 이름은 그냥 지나가.

4-2. 끝까지 따라가 보기: Workout의 "가슴" 버튼

app.js (줄임)

```
<button class="big" data-action="start" data-split="가슴">가슴</button>

function startWorkout(split) {
  const w = { id: Date.now().toString(36), split, start: new Date().toISOString(), end: null,
  exercises: [] };
  data.workouts.push(w); // ① data 바꾸기
  data.currentId = w.id;
  save(); // ② 저장
  Object.assign(ui, { screen: "workout", adding: true, ... }); // ③ ui 바꾸기 (Python의
  ui.update({...}))
  render(); // ④ 다시 그리기 → views["workout"] → workoutView()
}
```

4-3. 다시 그리기의 함정

render()는 #app 상자 안을 **통째로 새로** 만들어. 그래서 입력 칸에 글자를 치는 중에 render()가 불리면 **적던 글자가 사라져**. 그래서 sync.js는 동기화가 끝나도 바로 그리지 않고 refreshIfSafe()로 확인해:

sync.js

```
function refreshIfSafe() {
  if (["home", "workouts"].includes(ui.screen)) render();
  if (ui.screen === "nutrition" && !ui.mealForm && !ui.ratioPanel) render(); // 식사 추가 칸이 열려 있으면 안 그림
}
```

문제 — 먼저 종이에 예상하고, 정답은 맨 아래

Q4-1 어깨 날 운동 중, 두 번째 종목(번호 1)의 세트 완료 버튼은 HTML로 <button data-action="complete" data-i="1">야. 이걸 누르면 어떤 함수가 어떤 값으로 불릴까? Number() 가 없으면 무엇이 들어갈까?

Q4-2 Workout의 "가슴" 버튼을 누른 뒤 일어나는 일을 순서대로 4단계로 말해 봐. 마지막에 화면을 그리는 함수 이름은?

Q4-3 nutrition.js의 리스너에 case "start" 가 없는데, 가슴 버튼을 누르면 nutrition.js 리스너에서는 무슨 일이 일어날까?

Q4-4 식사 추가 칸에 "닭가"까지 쳤을 때 동기화가 끝났어. refreshIfSafe() 없이 그냥 render() 를 불렀다면?

5. 저장과 동기화 — 폰에서 인터넷까지

20분

🔗 "세트 완료 → 폰 저장 → Supabase"를 코드 이름으로 설명하기

5-1. save · load (5일차 복습)

app.js (줄임)

```
function save() {
  localStorage.setItem(STORAGE_KEY, JSON.stringify(data)); // 사전 → 글자 → 폰 보관함 (글자만 들어감!)
}
function load() {
  try {
    const saved = JSON.parse(localStorage.getItem(STORAGE_KEY)); // 글자 → 사전
    if (saved && Array.isArray(saved.workouts))
      return renameExercise({ ...EMPTY_EXTRAS(), ...saved }, "Incline bench press", "Smith IBP");
  } catch (e) { /* 깨졌으면 새로 시작 */ }
  return { workouts: [], currentId: null, custom: {}, ...EMPTY_EXTRAS() };
}
// EMPTY_EXTRAS() = { deleted: [], body: {}, meals: [], nutrition: null, hiddenGraphs: [], ... } ← 나중 버전에서 늘어난 칸들
```

{ ...EMPTY_EXTRAS(), ...saved }: 빈 칸들을 먼저 깔고, 저장본으로 덮어써. 그래서 v0.4-c 때 저장한 기록(체중 칸이 없음)을 열어도 data.body가 {}로 생겨서 에러가 안 나. (실행 셀 2-3 ①과 같은 원리)

실행 셀 5-1

```
const data = { body: { "2026-10-09": { w: 83.6 } } };
const text = JSON.stringify(data);
console.log(typeof text, text); // ① 보관함에 들어가는 모양

const back = JSON.parse(text);
console.log(back.body["2026-10-09"].w + 0.1); // ② ± 버튼으로 0.1 더하면?
const round1 = x => Math.round(x * 10) / 10;
console.log(round1(back.body["2026-10-09"].w + 0.1)); // ③ 앱에서 쓰는 방법
```

5-2. async / await — 인터넷 답 기다리기

JS	Python	뜻
async function f()	async def f():	기다리는 일이 있는 함수
await db.from(...).select()	await ...	답이 올 때까지 이 함수만 여기서 멈춤

JS	Python	뜻
throw error	raise error	에러 던지기 → 가장 가까운 catch로 점프
try { } catch (e) { }	try: ... except: ...	
const { data: rows, error } = await ...		답에서 data를 꺼내며 이름을 rows로 바꿈

실행 셀 5-2

```
const fakeDb = {
  select: () => new Promise(ok => setTimeout(() => ok({ data: [1, 2], error: null }), 300)),
};
async function run() {
  console.log("A");
  const { data: rows, error } = await fakeDb.select(); // 답 올 때까지 여기서 기다림
  console.log("B", rows.length, error);
}
run();
console.log("C");
```

5-3. syncNow()의 순서 — 지우기 → 올리기 → 받기

sync.js (빠대만)

```
async function syncNow() {
  if (!account.user || account.busy || !navigator.onLine) return; // 로그인 X · 이미 하는 중 · 인터넷 X → 안 함
  try {
    // ① 폰에서 지운 운동 → 인터넷에서도 지우기 (data.deleted)
    // ② 아직 안 올린 운동 올리기 (synced: false인 것, 하나 올릴 때마다 synced = true;
    save())
    // ③ 내가 만든 종목 올리기 (customDirty)
    // ④ 체중 · 식사 · 식단 설정 (syncNutrition) (synced: false · nutrition.dirty)
    // ⑤ 숨긴 그래프 목록 (syncAppSettings) (hiddenDirty)
    // ⑥ 받기 (downloadAll) - 다른 기기에서 한 것 가져오기
  } catch (error) {
    account.status = `동기화 실패 - 다음에 다시 시도해요 (...)`; // 표시가 그대로 남아 있으니 다음에 다시
  }
}
```

"아직 안 올림" 표시	붙는 곳	모양
synced: false	운동 하나, 체중 하루, 식사 하나 마다	여러 개 → 바뀐 것만 골라 올리고, 끊기면 이어서
customDirty	내 종목 목록 전체	통째로 하나 → 표시 하나
nutrition.dirty	식단 설정 (비율 · 속도)	통째로 하나
hiddenDirty	숨긴 그래프 목록	통째로 하나

올리기를 먼저, 받기를 나중에 하는 이유: 받기는 "인터넷에 없는데 폰에 '올라간 것'으로 표시된 건 다른 기기에서 지운 것"이라고 보고 지워. 순서가 반대면 방금 만든 기록을 지운 걸로 착각할 수 있어. 그래서 방금 올린 건 justUploaded로 한 번 더 보호해.

5-4. 서비스 워커 (sw.js) — "왜 새 버전은 두 번 열어야 보여?"

앱 열기 → 폰에 **보관된 파일**로 바로 보여 줌 (빠름, 인터넷 없어도 됨)

↓ 동시에

뒤에서 인터넷의 **새 파일**을 받아 보관함에 갈아 뒀

↓

다음에 열 때 새 파일이 보임. 단, APP_VERSION과 sw.js의 CACHE 이름을 같이 올리면 새 서비스 워커가 설치되면서 **자동 새로 고침**

문제 — 먼저 종이에 예상하고, 정답은 맨 아래

Q5-1 실행 셀 5-1의 ①~③을 예상해 봐. ②는 왜 83.7이 아닐까? 앱에서 이걸 어떻게 막고 있지?

Q5-2 실행 셀 5-2의 출력 순서를 예상해 봐. 왜 "C"가 "B"보다 먼저일까?

Q5-3 인터넷이 안 되는 곳에서 운동을 끝냈어. 다음 날 인터넷 되는 곳에서 앱을 열면 그 운동은 어떻게 될까? 어떤 표시 덕분이야?

Q5-4 sync.js에서 `const {{ data: rows, error }} = ...`처럼 **data**를 **rows**로 이름을 바꿔 꺼내는 이유는? (힌트: app.js에서 data는 이미 무엇의 이름이지?)

Q5-5 운동을 하나 올릴 때마다 `w.synced = true; save();`를 바로 하는 이유는? (10개 올리다 5번째에서 인터넷이 끊기면?)

Q5-6 코드를 고쳐서 올렸는데 APP_VERSION과 CACHE를 안 올렸어. 폰에서는 언제 새 버전이 보일까?

6. 핵심 규칙 3개 — 손으로 따라가기

15분

🔴 앱이 내놓는 숫자를 내가 손으로 계산해서 맞춰 보기 (현미경 바닥선)

6-1. 다음 무게 — nextWeight (노트북 ⑧)

규칙 순서가 곧 우선순위야. 위에서부터 처음 맞는 줄에서 **return** → 함수 끝. 거짓이면 다음 줄로.

실행 셀 6-1

```
function nextWeight(weight, reps, low, high, step, rpe) {
  if (weight === 0) return { weight: 0, reason: "맨몸 → 1회씩 더" };
  if (Math.min(...reps) >= high && rpe === 10) return { weight, reason: "굳히기" };
  if (Math.min(...reps) >= high) return { weight: weight + step, reason: "증량" };
  if (Math.max(...reps) < low) return { weight: Math.max(weight - step, 0), reason: "감량" };
  return { weight, reason: "같은 무게" };
}
// Bench press: 범위 6~8, 증량 5
console.log(nextWeight(85, [8, 8, 8, 8], 6, 8, 5, 9)); // ①
console.log(nextWeight(85, [8, 8, 8, 8], 6, 8, 5, 10)); // ②
console.log(nextWeight(85, [8, 7, 6], 6, 8, 5, null)); // ③
console.log(nextWeight(85, [5, 5, 4], 6, 8, 5, null)); // ④
```

6-2. 유지 칼로리 · 추천 섭취 — maintenanceInfo · targetKcal (노트북 ④ · ⑤)

nutrition.js의 식 (말로)

유지 칼로리 = 평균 섭취 - (하루 체중 변화 × 7700) ← 최근 14일, 체중 10일 · 식사 7일(오늘 제외) 이상 일 때
추천 섭취 = 유지 + 목표 kg/주 × 7700 ÷ 7 → 10kcal 단위 반올림

nutrition.js (줄임)

```
const kdays = [...].filter(d => d > start && d < today); // 먹은 날 (오늘 빼고 - 하루가 안 끝났으  
니)  
const meanKcal = kdays.reduce((s, d) => s + dayTotals(d).kcal, 0) / kdays.length;  
return { maintenance: meanKcal - perDay * KCAL_PER_KG, ... };  
  
function targetKcal(maintenance) {  
  return Math.round((maintenance + nutritionSettings().kgPerWeek * KCAL_PER_KG / 7) / 10) * 10;  
}
```

6-3. 운동 그래프의 점 — progressSeries

그날 그 종목의 가장 무거운 무게가 점 하나 (같은 무게면 횟수가 많은 세트). e1RM은 그날 세트 중 가장 높은 값을 그래프 아래 숫자로.

실행 셀 6-3

```
const round1 = x => Math.round(x * 10) / 10;  
const e1rm = (w, r) => round1(w * (1 + r / 30));  
const sets = [{ w: 85, r: 8 }, { w: 85, r: 9 }, { w: 80, r: 10 }]; // 그날 Bench press  
  
let top = null, best = 0;  
for (const s of sets) {  
  if (!top || s.w > top.w || (s.w === top.w && s.r > top.r)) top = s; // 그날 그래프 점  
  best = Math.max(best, e1rm(s.w, s.r)); // 그래프 아래 e1RM  
}  
console.log(sets.map(s => e1rm(s.w, s.r))); // ① 세트마다 e1RM  
console.log(top, best); // ② 점과 e1RM
```

문제 — 먼저 종이에 예상하고, 정답은 맨 아래

Q6-1 실행 셀 6-1의 ①~④를 예상해 봐. (Bench press 범위 6~8, 증량 5)

Q6-2 최근 14일 평균 섭취 2,200kcal, 체중이 하루에 -0.05kg씩 줄었어. 유지 칼로리는? 목표가 주당 -0.5kg이면 추천 섭취는? (계산 과정도)

Q6-3 Q6-2에서 체중이 오히려 하루 +0.05kg씩 늘었다면 유지 칼로리는 2,200보다 클까 작을까?

Q6-4 실행 셀 6-3의 ①, ②를 예상해 봐. 그래프의 점은 몇 kg이고, 그 아래 e1RM은 몇 kg으로 나올까?

7. 고칠 곳 찾기 — "내 손으로 1줄" 연습

15분

🔗 바꾸고 싶은 게 생기면 파일과 이름을 바로 찾기 (코드를 몰라도 설정은 고칠 수 있게)

앱은 노트북 ②처럼 바꿀 만한 숫자를 파일 맨 위 이름(설정)으로 빼 뒀어. 대부분은 그 한 줄만 고치면 돼.

파일 맨 위 설정들

app.js APP_VERSION · RPE_CHOICES · PART_ORDER · STEP_CHOICES

exerci EXERCISES (종목마다 part · range · step)
ses.js

nutrit DEFAULT_RATIO · RATIO_PRESETS · DEFAULT_KG_PER_WEEK · SPEED_CHOICES · INITIAL_MAINTENANCE ·
ion.js WINDOW_DAYS · MIN_WEIGHT_DAYS · MIN_KCAL_DAYS · KCAL_PER_KG · CHART_DAYS · MEAL_SLOTS

파일 맨 위 설정들

progre GRAPH_MIN_RECORDS · e1rm
ss.js

sw.js CACHE

문제 — 먼저 종이에 예상하고, 정답은 맨 아래

Q7-1 체중 그래프를 최근 60일로 → 어느 파일의 무엇?

.....

Q7-2 유지 칼로리를 최근 21일로 계산 → ?

.....

Q7-3 기록이 쌓이기 전 유지 칼로리를 2,600으로 → ?

.....

Q7-4 RPE 버튼에 6도 넣기 → ?

.....

Q7-5 새 종목 증량 단위 후보에 1.25kg 넣기 → ?

.....

Q7-6 Bench press 반복 범위를 5~7로 → ?

.....

Q7-7 탄단지 목표 버튼에 6:2:2 넣기 → ? (값은 어떻게 적어야 할까?)

.....

Q7-8 목표 속도 버튼에 -0.4kg 넣기 → ?

.....

Q7-9 기록이 2번 이상인 종목만 그래프 목록에 → ?

.....

Q7-10 버튼 파란색 바꾸기 → ?

.....

Q7-11 ⚠ 판단 문제: 끼니에 "운동 후"를 추가하려고 MEAL_SLOTS에 "운동 후"를 넣었어. 폰 화면에는 잘 나오는 데, 무엇이 실패할까? (힌트: Supabase 표를 만들 때 meals 표의 slot 칸에 무슨 규칙을 걸었지?)

.....

Q7-12 어떤 설정이든 고쳐서 올린 뒤에 같이 해 주면 좋은 것 2가지는?

.....

8. 조용한 버그 도감 + 말로 설명하기

10분

🎯 "에러가 안 났으니 맞다"가 아니라는 감각 + 오늘 전체를 내 말로

에러는 눈에 보여서 오히려 찾기 쉬워. 위험한 건 **에러 없이 조용히 틀리는 것**이야. 지금까지 우리가 만난 것들:

상황	에러?	결과	막는 법
사전을 그대로 localStorage.setItem	없음	"[object Object]" 로 저장 → 기록 망가짐	JSON.stringify
reduce에 , 0 빠짐	없음	"10-081130" 글자 이어 붙이기	시작값 , 0
map 뒤 join("") 빠짐	없음	버튼 사이에 쉼표 ,	.join("")
사용자 글자를 그대로 HTML에	없음	글자가 HTML로 해석됨	escapeHtml
빈 목록 [] 를 if로 검사	없음	JS에서는 참	.length
83.6 + 0.1	없음	83.69999999999999	round1
0 70	없음	0인데 70이 됨	??
오늘 칼로리를 평균에 포함	없음	평균 ↓ → 유지 칼로리 ↓	d < today
dirty 표시 없이 동기화	없음	폰에서 바꾼 게 인터넷 것에 덮여 사라짐	hiddenDirty = true
스크립트 순서가 틀림	없음	체중 · 식사가 안 올라감	index.html 순서 + typeof ... === "function"

마지막: 말로 설명하기 (종이에 3~5줄씩)

- 1. 세트 완료 버튼을 눌렀을 때부터 그 세트가 **Supabase의 sets 표**에 들어갈 때까지. (힌트 단어: data-action · completeSet · save · render · finishWorkout · syncNow · synced · uploadWorkout)
 - 2. 유지 칼로리가 "초기값 2,750"에서 "내 기록으로 계산"으로 바뀌는 조건과 계산식.
 - 3. GitHub에 코드를 고쳐 올린 뒤 **폰에 새 버전이 보이기까지**. (힌트: Vercel · sw.js · CACHE · 두 번 열기)
- 세 개 다 막힘없이 쓰면 오늘 복습 목표 달성이야. 막힌 단어가 있으면 그 장을 다시 보고, 다음에 같이 이야기하자.

✓. 정답

출력 모양: 문자열 목록은 ['a', 'b'], 사전은 { w: 80, r: 10 }로 적었어. 크롬 콘솔은 (2) ['a', 'b'], {w: 80, r: 10} 처럼 조금 다르게 보일 수 있어.

Q1-1 (가) data (나) ui (다) ui (라) data

Q1-2 (나) 탭과 (다) 펼친 그래프 — ui는 저장하지 않아서 처음 상태(식단 탭, 다 접힘)로 돌아가.

Q1-3 (가) progress.js (나) sync.js의 authError (다) exercises.js의 EXERCISES (라) style.css의 --accent

Q2-1

```
[8, 7, 10]
2
{ w: 80, r: 10 }
true false
85
```

④ 7회짜리가 있어서 some(r < 8)은 true, 그래서 every(r ≥ 8)은 false. ⑤ ... 없이 Math.max(list) 면 NaN. (크롬 콘솔에서는 (3) [8, 7, 10] 처럼 개수가 앞에 붙어 보일 수 있어)

Q2-2

```
['10-08', '10-09']
10-08=2200, 10-09=1130
3330
10-081130
['Dips', 'Pull up']
```

④ 시작값이 없으면 첫 칸("10-08", 날짜 글자)이 시작값이 돼. 글자 + 숫자 = 글자 이어 붙이기 → "10-08" + 1130. 첫 칸 자리의 2200은 아예 더해지지도 않아.

Q2-3

```
['deleted', 'body', 'workouts', 'custom'] { A: 1 }
30
70 0 70
null
```

① 순서는 처음 나온 자리대로, custom은 **마지막에 쓴 { A: 1 }**이 이김. ③ ?? 는 null/undefined일 때만 바꾸고, || 는 0 · "" 같은 거짓 값이면 다 바꿔. 그래서 0이 의미 있는 값일 땐 ?? . ④ rec 이 null이면 && 뒤는 보지도 않고 null (에러 안 남).

Q2-4

```
거짓 거짓 거짓
참 참 참
거짓
```

② JS에서는 빈 목록 [], 빈 사전 {}도 참이야 (Python과 반대). "0"은 글자라서 참. ③ [].length 는 0 → 거짓. 그래서 빈지 볼 때는 !items.length .

Q2-5 top = max([s["w"] for s in sets if s["r"] >= 8])

Q3-1

```
2
<button ...>등 26세트</button><button ...>가슴 44세트</button>
<main><button ...>등 26세트</button>,<button ...>가슴 44세트</button></main>
```

① map은 목록(글자 2개짜리)을 돌려줘. ③ 목록을 글자 안에 바로 넣으면 쉼표로 이어져서, 화면에 버튼 사이 쉼표,가 보여.

Q3-2

```
<b><i>닭</i> 가슴살</b>
<b>&lt;i>닭</i>&lt;/b> 가슴살</b>
```

화면에서 ①은 **닭가슴살**(닭이 기울어짐 — HTML로 해석), ②는 **<i>닭</i> 가슴살**이 글자 그대로 보여.

Q3-3 nutritionView() — nutrition.js에 있어. (render는 app.js에 있지만 다른 파일의 화면 함수를 이름으로 불러)

Q3-4 에러는 안 나. 앱을 열자마자 하는 동기화 때 syncNutrition이 아직 없어서 typeof ... === "function" 이 거짓 → 체중 · 식사 · 식단 설정을 건너뛰. 조용히 안 올라가는 버그 (실제로 있었던 일).

Q3-5	3군데: 라이트 모드 : root, 폰이 다크 모드일 때(@media), 다크로 고정했을 때([data-theme="dark"]). 라이트만 바꾸면 다크 모드에서는 여전히 파랑.
Q4-1	completeSet(1).el.dataset.i는 글자 "1" 이라서, Number가 없으면 completeSet("1") → 안에서 current().exercises["1"]은 우연히 동작하지만, ui.edit.ex === i 같은 비교(1 === "1"은 false)에서 틀려. 그래서 처음부터 숫자로 바꿔 뒤.
Q4-2	① click 리스너가 data-action="start", data-split="가슴"을 읽어 startWorkout("가슴") 호출 ② data.workouts에 새 운동 추가 + currentId ③ save() ④ ui.screen = "workout" 후 render() → views["workout"] = workoutView()
Q4-3	아무 일도 없어. switch에 맞는 case가 없으니 그냥 끝나. 리스너는 여러 개여도 각자 자기 이름만 처리해.
Q4-4	#app 안이 통째로 새로 그려져서 입력 칸도 새로 생김 → "닭가"가 사라져. 그래서 식사 칸이 열려 있으면 안 그리게 막아 둔 것.
Q5-1	<pre>string {"body":{"2026-10-09":{"w":83.6}}}</pre> <pre>83.69999999999999</pre> <pre>83.7</pre> 컴퓨터는 0.1 같은 소수를 2진수로 정확히 못 담아서 아주 작은 오차가 생겨. 앱은 round1($\times 10 \rightarrow$ 반올림 $\rightarrow \div 10$)으로 소수 첫째 자리에 맞춰.
Q5-2	<pre>A</pre> <pre>C</pre> <pre>B 2 null</pre> await는 run 함수만 멈추고, 함수 밖의 코드(C)는 계속 진행돼. 0.3초 뒤 답이 오면 B. 앱에서 동기화하는 동안에도 화면을 계속 쓸 수 있는 이유.
Q5-3	운동에 synced: false가 남아 있어서, 앱을 열 때 하는 syncNow()가 그 운동을 올려. 기록은 항상 폰에 먼저 저장되니까 사라지지 않아.
Q5-4	app.js의 전역 이름 data가 이미 "앱의 모든 기록"이야. 꺼낸 걸 그대로 data라고 부르면 헷갈리거나 덮어쓸 위험 → rows로 바꿔서 꺼냄.
Q5-5	5번째에서 끊기면 1~4번은 이미 synced = true로 저장돼 있어서, 다음에 5번부터 이어서 올려. 끝에 한 번만 저장하면 1~4번을 또 올리게 돼.
Q5-6	자동 새로 고침이 없어. 첫 번째로 열 때는 보관된 옛 파일, 그사이 뒤에서 새 파일을 받아 두고 → 두 번째로 열 때 새 버전.
Q6-1	<pre>{ weight: 90, reason: '증량' }</pre> <pre>{ weight: 85, reason: '균히기' }</pre> <pre>{ weight: 85, reason: '같은 무게' }</pre> <pre>{ weight: 80, reason: '감량' }</pre> ① 다 8회(최대) + RPE 9 → +5 ② 다 8회지만 RPE 10 → 균히기(두 번째 줄이 먼저 걸림) ③ 최소 $6 < 8$, 최대 $8 \geq 6$ → 같은 무게 ④ 최대 $5 < 6$ → -5
Q6-2	유지 = $2,200 - (-0.05 \times 7,700) = 2,200 + 385 = 2,585\text{kcal}$. 추천 = $2,585 + (-0.5 \times 7,700 \div 7) = 2,585 - 550 = 2,035 \rightarrow 10\text{kcal}$ 반올림 2,040kcal .
Q6-3	작아 ($2,200 - 385 = 1,815$). 2,200을 먹었는데 체중이 늘었으면 몸이 쓰는 건 그보다 적다는 뜻.
Q6-4	<pre>[107.7, 110.5, 106.7]</pre> <pre>{ w: 85, r: 9 } 110.5</pre> 점 = 85kg (85kg 세트가 둘인데 횟수가 많은 85×9), 아래 e1RM = 110.5kg ($85 \times (1 + 9/30) = 85 \times 1.3$).
Q7-1	nutrition.js CHART_DAYS = 60
Q7-2	nutrition.js WINDOW_DAYS = 21

Q7-3	nutrition.js INITIAL_MAINTENANCE = 2600
Q7-4	app.js RPE_CHOICES = [6, 7, 8, 9, 10]
Q7-5	app.js STEP_CHOICES = [1, 1.25, 2, 2.5, 4, 5]
Q7-6	exercises.js "Bench press": { part: "가슴", range: [5, 7], step: 5 } (노트북 ②도 같이)
Q7-7	nutrition.js RATIO_PRESETS = [[50, 30, 20], [40, 30, 30], [60, 20, 20]] — 비율은 %로, 합이 100 이 되게.
Q7-8	nutrition.js SPEED_CHOICES 에 -0.4 넣기 (순서 맞춰서)
Q7-9	progress.js GRAPH_MIN_RECORDS = 2
Q7-10	style.css --accent 3군데 (Q3-5)
Q7-11	동기화 가 실패해. meals 표에 check (slot in ('아침', '점심', '저녁', '간식')) 규칙이 있어서 "운동 후" 식사는 Supabase가 거절 → "동기화 실패". 화면 설정(MEAL_SLOTS)과 인터넷 표 규칙(migration) 을 같이 바꿔야 해. 이런 건 혼자 1줄로 고치기보다 나랑 같이.
Q7-12	① app.js APP_VERSION과 sw.js CACHE 이름 올리기 (폰이 바로 새로 고침) ② 폰에서 홈 맨 아래 버전 번호로 확인. (노트북 규칙과 연결된 값이면 노트북도 같이)

막힌 곳(☆)은 다음 세션에 가져와. 거기서부터 같이 보자.